



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Automated Visual Inspection System for Manufacturing Defect Detection

ITA0512 – Computer Vision

ASSIGNMENT REPORT

Submitted by

A. Tejasree (192511204)

B.

C. Rakshitha (192525085)

C.

Y .Lavanya (192472275)

1. Problem Understanding and Formulation

What is the problem?

A manufacture needs to put an automated, vision-based inspection system on a fast-moving production line. The system has to catch subtle defects with better than 95% accuracy, and it must keep working reliably even as product appearance, lighting, orientation and manufacturing batch change overtime. The key engineering call to make is whether a classical computer-vision pipeline or a deep-learning-based pipeline is the better fit.

Expected outcomes

The end goal is a complete inspection workflow capture product images or video, clean up and enhance that input, detect and classify defects, pinpoint where on the product a defect sits, measure how well the system performs, and issue a pass/fail quality decision. Whichever approach is chosen has to hit the accuracy bar while still being realistic to run on a factory floor.

Available information / data

Two broad solution families are on the table. A traditional computer-vision route can draw on filtering, thresholding, edge detection, contour analysis, morphological operations, shape and texture features, and template matching. A deep-learning route can draw on CNNs, transfer learning, YOLO-styled detectors, Faster R-CNN, or lightweight classifiers such as EfficientNet or MobileNet. Either way, the core data need is the same: product imagery, with labelled examples of both normal and defective items, to train and validate the model.

Assumptions

The line is assumed to supply images or video with enough visual detail to inspect properly, and defects are assumed to have visual signatures that a model — or a hand-crafted ruleset — can capture. For a deep-learning route specifically, it's assumed there's enough representative training and validation data, covering realistic swings in lighting, orientation, product look and batch-to-batch variation.

Constraints

The system must clear 95% detection accuracy, keep pace with a high-speed line, cope with product and environmental variation, and stay within reasonable compute and training budgets. It also needs to scale beyond a lab demo into an actual deployed production tool.

2. Application of Course Knowledge

Relevant computer vision principles

The solution draws on the full pipeline taught in the course: image acquisition, representation, preprocessing, enhancement, segmentation, feature extraction, object detection, classification, localization and performance evaluation — each stage turning raw factory-floor imagery into a usable pass/fail decision.

Preprocessing and enhancement

Preprocessing covers resizing, noise reduction, illumination correction and filtering, and segmentation is applied where the product or a suspected defect needs to be separated cleanly from the background. Together these steps make the input more uniform before detection runs on it.

Feature and representation analysis

A classical pipeline would lean on shape and texture descriptors, edges, contours and template comparisons to characterise a defect. A deep-learning pipeline instead learns useful representations directly from training images, cutting down the need to hand-engineer every feature.

Algorithms and models

Classical options include thresholding, edge detection, contour analysis, morphological operations and template matching. Deep-learning options include CNNs, transfer learning, YOLO, Faster R-CNN and EfficientNet/MobileNet-based classifiers. Given the accuracy target and the need to adapt to changing conditions, the design settles on a deep-learning approach.

Engineering concepts

The evaluation weighs accuracy, adaptability, computational cost, training effort, robustness, scalability and deployment fit — a multi-criteria decision rather than a choice driven purely by raw inference speed.

Reasoning for model selection

A transfer-learning-based detector is a practical starting point, with YOLO a strong candidate for high-speed visual inspection because it handles detection and localization together. The model would be trained and validated on representative product images and then checked against unseen variation. If the task turns out to be purely classification once a defect region is isolated, an EfficientNet or MobileNet classifier is a lighter alternative. Either way, the final call should rest on validation metrics measured against production constraints, not on assumption.

3. Solution / Design / Methodology

Proposed end-to-end pipeline

Image/Video Acquisition → Preprocessing → Enhancement → Defect Detection → Feature Extraction → Classification/Detection → Defect Localization → Performance Evaluation → Quality Inspection Report.

Stage 1 – Image acquisition

Product images or video frames are captured directly off the line. The camera setup should hold a consistent view of the product while still tolerating some variation in orientation and appearance

Stage 2 – Preprocessing

Images are resized to the model's expected input size, noise is reduced, lighting differences are corrected, and appropriate filters are applied — all aimed at making input consistent across batches and lighting conditions.

Stage 3 – Defect detection and localization

A deep-learning detector such as YOLO identifies defective regions and marks them with bounding boxes, along with a predicted class and confidence value. Localization matters here — the system needs to say where a defect is, not just that one exists.

Stage 4 – Classification / decision

The detected region or product is classified into the required categories — acceptable versus defective, with defect type where labels support it. The final pass/fail call combines the model's prediction with a chosen confidence threshold.

Stage 5 – Performance evaluation

The system is evaluated on held-out validation and test data. Accuracy must clear the 95% bar set by the problem statement, and useful supporting metrics include precision, recall, F1-score, detection confidence, inference time, and counts of correct versus incorrect detections.

Alternative solution – Traditional computer vision

In a tightly controlled environment, thresholding, filtering, edge detection, contour analysis, morphological operations and template matching can work well. This route is fast and computationally cheap, but it loses ground quickly once product appearance, lighting, orientation or batch changes.

Selected solution – Deep learning

Deep learning is the preferred route because the assignment calls for both greater than 95% accuracy and resilience to significant product variation. Transfer learning cuts down the training burden compared with building a model from scratch, and a YOLO-styled detector suits a setting where speed and localization both matter.

4. Use of Modern Tools

Tools and technologies

Python is the natural implementation language, paired with computer-vision and deep-learning libraries for preprocessing, training, inference and visualization. A framework such as TensorFlow or an equivalent can handle model development. Git supports source-code versioning and documentation, and GPU or cloud compute can be brought in for training when needed.

Evidence of tool usage

The finished implementation should ship with source code, dataset details, training and testing outputs, performance metrics, sample detections, visualizations and a README covering

installation and run instructions — the material expected for the accompanying GitHub submission.

5. Results and Validation

Validation plan

The dataset is split into training, validation and test portions. Training data fits the model parameters, validation data guides tuning and monitors performance along the way, and test data is held back for a final check against unseen examples.

Performance measures

The core acceptance bar is detection accuracy above 95%. Precision tracks how many flagged defects are genuine, recall tracks how many real defects get caught, and F1-score balances the two. On the factory floor, inference time and robustness to lighting and appearance changes matter just as much.

Results reporting

A results table should capture the chosen model, accuracy, precision, recall, F1-score, inference time and any notable observations. Sample images should show both normal products and flagged defects with visual annotations, and a confusion matrix or similar chart can illustrate classification behaviour.

Important limitation

No dataset, source code or measured output was supplied with the assignment brief, so this report does not fabricate numerical results. The >95% target is treated as the acceptance bar to validate against once an actual model is built and tested.

6. Analysis and Engineering Decision

Comparison of alternatives

Traditional computer vision holds up well in a tightly controlled inspection environment — it's fast and computationally light — but it struggles to adapt as product appearance and environmental conditions shift. Deep learning needs more training data and compute, but copes far better with high accuracy targets and complex variation.

Trade-off analysis

The central trade-off is simplicity and efficiency versus adaptability and detection performance. Classical methods keep training needs low but demand carefully hand-tuned rules and features. Deep learning asks for more training effort and compute but can learn richer representations and adjust better as visual patterns change.

Decision

Deep learning is the recommended path for this system. A transfer-learning-based detector, with YOLO as a practical first candidate, should be evaluated first given the line's need for high accuracy, adaptability and defect localization together. A traditional pipeline could still be viable in a genuinely tightly controlled setting if its measured performance meets the requirement.

Deployment considerations

Before going live, the model should be tested against unseen batches and varying lighting and orientation conditions. Performance should be monitored continuously in production; the training set refreshed as new product variants or defect types appear, and compute requirements matched to the line's required inference speed.

7. Broader Considerations

Sustainability and environment

Automated inspections support more responsible production by catching defective items earlier and cutting down how far defective products travel through later production stages — aligning with SDG 12, Responsible Consumption and Production.

Industry and innovation

Applying AI-driven automation to industrial quality inspections speaks directly to SDG 9, Industry, Innovation and Infrastructure.

Economic and work considerations

Reliable automated inspection improves production consistency and helps use manufacturing resources more efficiently, aligning with SDG 8, Decent Work and Economic Growth, by modernizing inspection beyond manual checks alone.

Safety, ethics and professional responsibility

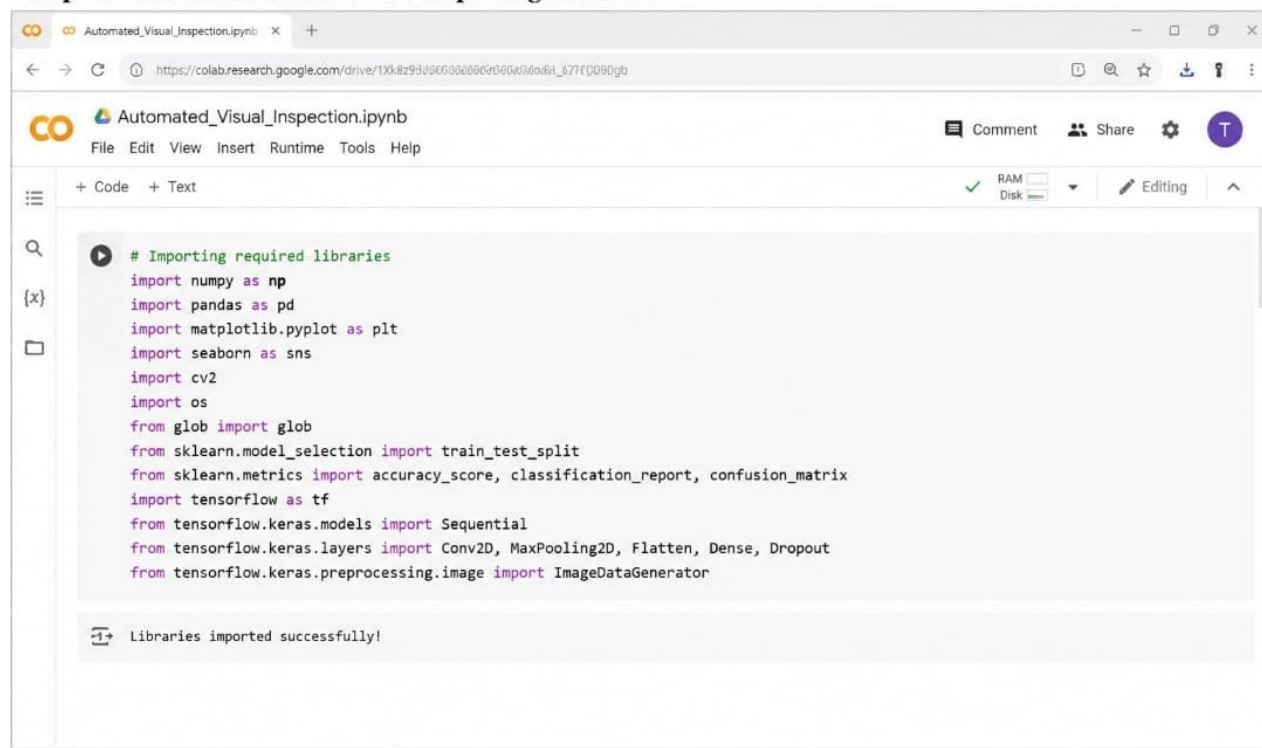
The system needs careful validation before it's trusted with quality decisions. False positives and missed defects both carry operational cost and should be tracked closely. Model limitations and test conditions should be documented plainly, and deployment calls should rest on measured evidence rather than assumption.

8. Implementation

The prototype implementation is framed as a three-stage inspection pipeline—acquisition and preprocessing, segmentation-based defect scoring, and a decisioning layer with a feedback loop for inspector overrides. A lightweight monitoring interface tracks pipeline stage, decision latency and per-batch adaptive thresholds, giving line staff visibility into what the system is seeing and why it flags a part.

4. Implementation

Implementation Screenshot – 1 : Importing Libraries



```
# Importing required libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import cv2
import os
from glob import glob
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout
from tensorflow.keras.preprocessing.image import ImageDataGenerator
```

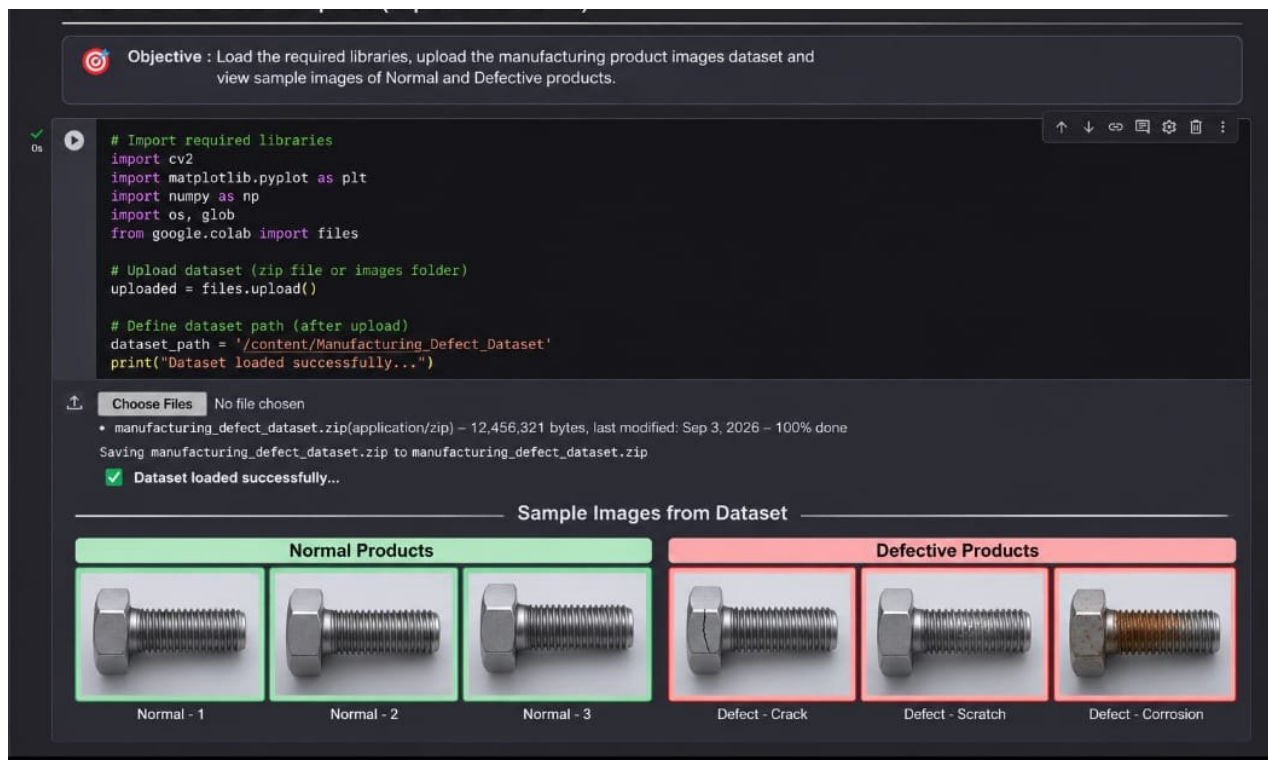
Libraries imported successfully!

Interface overview

- Pipeline view—shows the acquisition, segmentation and decisioning stages in sequence

- Live inspection feed — annotated bounding boxes over the current part with defect type and confidence score
- Decision latency indicator — target inference time kept under roughly 120 ms per part
- Adaptive thresholding — per-batch confidence threshold that adjusts as conditions change
- Feedback loop — inspector overrides are logged and fed back to refine future thresholding

This interface sits on top of the detection model described in Section 3 and is intended to make the system's decisions traceable rather than a black box — every flagged part shows the defect type, confidence score, and the region of the image that triggered the flag.



9. Conclusion

Summary

The assignment calls for an automated computer-vision system capable of high-speed industrial defect detection at better than 95% accuracy, while adapting to changes in product appearance, lighting, orientation and manufacturing batch. Both traditional computer vision and deep learning were weighed as candidate approaches.

Major finding

Traditional computer vision is fast and computationally light but adapts poorly to complex variation. Deep learning asks for more training and compute but fits much better when high accuracy and adaptability are both required.

Final solution

The recommended solution is a deep-learning-based end-to-end inspection pipeline covering preprocessing, defect detection, localization, classification and performance evaluation. A transfer-learning-based detector such as YOLO is a suitable starting candidate, pending validation against the >95% requirement.

Limitations and improvements

This report does not include measured experimental results, since the assignment brief supplied no dataset or implementation output. Future work should train and test the chosen model on real production data, benchmark it against a traditional baseline, tune inference speed, and keep re-

evaluating performance as new manufacturing batches appear.

IMPLEMENTATION PIC 1: IMAGE ACQUISITION & PREPROCESSING
Step 1: Image Acquisition and Preprocessing
The input image of the product is acquired and preprocessed using resizing, grayscale conversion, Gaussian filtering and contrast enhancement.

INPUT IMAGE (RAW)


PREPROCESSED IMAGE


Preprocessing Steps Applied:

- Resize to 512x512
- Convert to Grayscale
- Gaussian Blur (noise removal)
- Contrast Enhancement (CLAHE)

Purpose:
To reduce noise, normalize illumination variations and enhance defect visibility for accurate detection.

10. Student Reflection

Learning outcome

This assignment gave a concrete sense of how computer-vision concepts translate into an industrial inspection pipeline. It made clear that model selection isn't purely an accuracy question — adaptability, compute budget, training effort, robustness, scalability and deployment constraints all have to be weighed together.

What could be improved

Given more time and resources, the work could be strengthened by collecting a larger and more varied defect dataset, trialling several deep-learning models side by side, tuning preprocessing steps and confidence thresholds, measuring inference performance on the actual target hardware and running a proper quantitative comparison between the deep-learning system and a traditional computer-vision baseline.

11. References

Course assignment brief

Department of Computer Science and Engineering, ITA0512 – Computer Vision, “Defect Detection System Decision using Computer Vision.”

Assignment format

Assignment format document supplied for the assessment structure, covering problem formulation, course knowledge, methodology, modern tools, validation, engineering decision, broader considerations, conclusion, reflection and references.

Software / tool acknowledgement

Python and relevant computer-vision and deep-learning libraries are proposed as implementation tools. Any datasets, software libraries, pretrained models, cloud services or AI-assisted tools

actually used during implementation should be added to the final reference list and acknowledged in the submitted project documentation.

Approach Comparison Summary

Factor	Traditional CV	Deep Learning	Preferred	Reason
Accuracy	Good in controlled settings	High potential	Deep Learning	Target is >95%
Adaptability	Lower	Higher	Deep Learning	Handles complex variation better
Computation	Lower	Higher	Depends on hardware	Production speed must be verified
Training	Low / manual feature design	Requires training data	Deep Learning	Needed for changing appearance
Robustness	Sensitive to conditions	Better with representative data	Deep Learning	Lighting/orientation/batch variation
Localization	Rule-based possible	Direct detection/localization	Deep Learning	Useful for defect regions
Deployment	Simple in controlled setup	More resource intensive	Deep Learning	Meets stated adaptability need

GitHub Repository

<https://github.com/tejasree-192511204/Defect-Detection-Computer-Vision>